

ASSIGNMENT 1

CMSC 451 (Section 0201, Fall 2024)

Due by 11:59 pm on Wednesday, September 18. Submit your solutions via Gradescope.

0. *Collaboration.*

Indicate that you understand the course collaboration policy and have followed it when working on this assignment. List the other students with whom you discussed the problems, or else indicate that you did not discuss any problems with your classmates.

If you consulted any resources other than the textbook to help you understand how to approach a problem, list them at the end of the solution of that problem.

I understand the course collaboration policy, and I collaborated with Omkar Pathak, Jason Zhong, and Shrill Shah.

1. *Asymptotic growth.* [20 points]

For each of the following pairs of functions $f(n)$ and $g(n)$, indicate whether $f(n)$ is in O , o , Ω , ω , and/or Θ of $g(n)$. (You should either write true or false for each of these five possibilities, or give an exhaustive list of the relationships that hold.) Explanations are not required, but can be provided for possible partial credit.

- (a) $f(n) = 2^n$, $g(n) = n^2$
- (b) $f(n) = 2^{2023n}$, $g(n) = 2^{2024n-2025}$
- (c) $f(n) = n^3 - 2024n^2$, $g(n) = \sum_{k=1}^n k(k+1)$
- (d) $f(n) = (\log_2 n)^n$, $g(n) = n^{\log_2 n}$
- (e) $f(n) = n^{\pi/4}$, $g(n) = n^{\cos n}$

Solution.

- (a) O : False, o : False, Ω : True, ω : True, Θ : False
- (b) O : True, o : True, Ω : False, ω : False, Θ : False
- (c) O : True, o : False, Ω : True, ω : False, Θ : True
- (d) O : False, o : False, Ω : True, ω : True, Θ : False
- (e) O : False, o : False, Ω : False, ω : False, Θ : False

□

2. *Gale-Shapley algorithm.*

Consider the following instance of the stable matching problem, where each list indicates the ranked order of the indicated student/employer (from most preferred to least preferred).

Student preferences	Employer preferences
student 1: G, D, H, F, A, C, E, B	employer A: 4, 6, 2, 7, 8, 1, 3, 5
student 2: F, A, C, D, B, G, E, H	employer B: 1, 6, 5, 8, 2, 7, 4, 3
student 3: A, F, E, C, D, H, G, B	employer C: 6, 5, 3, 1, 8, 7, 2, 4
student 4: E, H, B, G, D, F, C, A	employer D: 3, 6, 1, 7, 4, 8, 5, 2
student 5: B, F, D, H, G, A, C, E	employer E: 7, 6, 8, 1, 2, 3, 4, 5
student 6: F, A, H, G, C, B, D, E	employer F: 7, 6, 5, 1, 3, 8, 2, 4
student 7: G, A, E, C, H, B, F, D	employer G: 4, 7, 8, 3, 2, 1, 6, 5
student 8: G, E, C, D, B, H, A, F	employer H: 4, 7, 1, 8, 6, 5, 3, 2

- (a) [10 points] Run the Gale-Shapley algorithm on this instance (with employers making offers to students). List which employer each student is paired with. (*Note:* You should either run the algorithm by hand, or implement it in a programming language of your choice. You may *not* use someone else's implementation.)
- (b) [10 points] Does this instance have a unique stable matching? Explain why or why not.

Solution.

- (a) I ran the algorithm by hand and got

{2: A, 1: B, 5: C, 3: D, 8: E, 6: F, 7: G, 4: H}

- (b) **No.** As proved in class, all outputs from the Gale-Shapley algorithm are stable. However, considering the case where students making offers to employers, we get the following output

{1: D, 2: A, 3: C, 4: H, 5: B, 6: F, 7: G, 8: E}

and this is clearly not the same output as in part A. Therefore, this instance does not have **unique** stable matching.

□

3. *Stable matching for a class project.* [15 points]

Suppose we want to assign partners in a class of $2n$ students. Each student provides a ranking of all other $2n - 1$ students (ties are not allowed). In this context, a *stable matching* is an assignment of partners so that no two students who are not partners would both prefer to work with each other rather than with their assigned partner. Show that a stable matching need not exist by giving an input instance (i.e., a set of rankings) and proving that no stable matching is possible for that instance. Try to make the value n as small as possible.

Solution. We can show that a stable matching doesn't exist with just 4 students ($n = 2$). Consider the following rankings:

Student A: B, C, D

Student B: C, A, D

Student C: A, B, D

Student D: A, B, C

First, since there are 4 students, there are only 3 possible ways to make 2 pairs.

1) Suppose A and B are paired. Therefore, C and D are paired. However, B would rather be with C than A and C would rather be with B than D.

2) Suppose A and C are paired. Therefore, B and D are paired. However, A would rather be with B than C and B would rather be with A than D.

3) Suppose A and D are paired. Therefore, B and C are paired. However, A would rather be with C than D and C would rather be with A than B.

Therefore, no stable matching exists for this instance with $n = 2$. □

4. *Fast path finding with a degree lower bound.* [20 points]

Suppose you are given an n -vertex graph in which each vertex has degree at least k . Give an algorithm running in time *independent of n* that finds a simple path in G of length k . Prove that your algorithm is correct and analyze its asymptotic running time as a function of k . For full credit, your algorithm should run in time $O(k^2)$.

Algorithm.

SimplePath(G)

- Let v be a random vertex in the graph, and set $curr = v$.
- Initialize arr to be a vertex-indexed array that contains one element, the element v .
- Initialize P to be a list that contains one element, the element v .
- **While** the length of $P < k$
 - **For** all neighbors of $curr$
 - * Check if this neighbor is in arr , the vertex-indexed array.
 - * **If** the vertex is in path, **continue**.
 - * **Else**, denote this next node in the path $next$.
 - Add $next$ to arr and to P , both of which are constant time operations.
 - Set $curr = next$, which is also a $O(1)$ time operation.
- Return the list P , which represents the simple path of length k in G .

The outer while loop runs for k iterations, each iteration also runs in worst case $O(k)$ time for the following reason: since arr is a vertex-indexed array, checking if a node u is in arr is an $O(1)$ operation. Note that for any node $curr$, we are guaranteed to find a node to be the next node in a simple path that is not already in arr in $O(k)$ time because every node has degree at least k , and there are guaranteed to be less than k nodes in arr — otherwise we would have found a simple path of length k already and exited the while loop. This scales with k rather than n because for any given node, we can find the next node in the path in k constant time checks because we are guaranteed that each vertex has degree at least k , and it is impossible for k of these neighbors to be in set because otherwise we would have already found our path.

Therefore, the time complexity of this algorithm is $O(k * k) = O(k^2)$, as desired.

□

5. *Orienting edges to avoid sinks.* [25 points]

Let $G = (V, E)$ be an undirected, connected graph. Design an algorithm to determine whether it is possible to direct the edges of G such that in the resulting directed graph, the outdegree of every vertex is at least 1. If it is possible, then your algorithm should show a way to do this. Prove that your algorithm is correct and analyze its running time. For full credit, your algorithm should run in time $O(|V| + |E|)$.

Algorithm.

I immediately realized that in the case of the undirected, connected graph having N edges and $N - 1$, it is not possible to direct the edges such that the outdegree of every vertex is at least 1. Note that intuitively, there are only $N - 1$ edges and N nodes; therefore, it cannot be possible that each node has an edge going out of it. One node will have an outdegree of 0. However, if the number of edges are greater or equal to the number of nodes, it is possible, and we proceed the following algorithm.

PossibleGraph(G):

- Let $T = (V, E)$ be the spanning tree generated from **SpanningTree**(G)
- Let v_0 be the starting node, $V' = V$, and $E' = E$
- Initialize q to be a queue with one element v_0
- **While** q is not empty
 - Now pop from the queue, and set this element to v_0
 - **For** all arbitrary vertices u such that E contains the edge (u, v_0) and (v_0, u)
 - * Remove this edge (u, v_0) from E'
 - * Push u to q
- **For** every edge e in E , the original set of edges
 - **If** e is of the form (v_0, u) , where u is an arbitrary vertex
 - Add e to E'
 - **Break**

Return $T' = (V', E')$, the new directed graph

SpanningTree(G):

- Let T be a graph with one vertex, with $pr[s] = \emptyset$
- **While** $\exists$ an edge (u, v) joining vertex u in T to a vertex v not in T
 - Add v and the edge (u, v) to T
- Set $pr[v] = u$

Proof of correctness:

We must show that this algorithm correctly orients the edges in a graph such that there are no sinks. It is important to note the edge case such that if there are less edges than vertices while still being an undirected, connected graph, the problem is impossible.

To prove correctness, the algorithm starts by generating a spanning tree from the graph G using the **SpanningTree**(G) function that contains all vertices in the original graph. We then take a random vertex in the graph, and begin a breadth-first search. At each vertex, remove all incoming edges to v_0 from E' which is originally set to the original set of edges. From there, add all neighbors of v_0 to the queue until all vertices in the graph have been visited. From here, we are guaranteed that all vertices except for the starting vertex have an outdegree of at least 1. To counter this, iterate through all the edges, and direct an edge from v_0 to any arbitrary edge u , and we are done. We have created a directed graph that avoids sinks. Note that the problem asks us to return True or False, and we return the graph, if desired.

For the run time, generating the spanning tree is $O(V + E)$. The BFS is also $O(V + E)$. Therefore, we have $O(2(V + E))$, which is still $O(V + E)$.

□